

PART 3 OF 5. BUILD A PREVIEW YOU CAN CHECK AGAINST THE TRACKER.

Run it by hand before you send anything

Before you turn on messages, run the script in preview mode. It should show who would receive each message, which work item it refers to, and the proposed text. Compare those details with the tracker before anyone else sees them.

This is a dry run: the script reads data and produces a local report without sending messages or changing work items. You'll use a fictional test set first, then review approved live data. By the end, you'll have a preview, a record of the problems you found, and a checklist for a small pilot.

1 Know what you're checking before you run it

Use the rule sheet from Part 2. For the exercise here, an active item is overdue when its due date is before the report date. The second list contains active items due today through seven days later, inclusive, with no state change for at least fourteen days.

That second condition needs reliable state-change history. A comment can change a "last modified" timestamp without changing the state. If history is missing, put a due-soon item in the review list. An overdue item doesn't need state history to qualify.

For every proposed message, the preview should show:

- ☐ Source, stable item ID, title, and link.
- ☐ Owner account ID and verified recipient account ID.
- ☐ Current state, due date, and the report date and time zone.
- ☐ Reason for the message, including state-change evidence when required.
- ☐ Exact proposed message and a clear "not sent" label.

EXAMPLE

Fictional preview: DEMO-101, "Update setup instructions," owner Alex Example, recipient alex-example, In progress, due September 21. Report date: September 22, 2026, Los Angeles. Reason: overdue by one calendar day. Message: "Hey Alex, DEMO-101 is listed as In progress with a September 21 due date. Does the tracker need an update?" Delivery: not sent.

WHY THIS MATTERS: each field in the preview is one thing you can check against the tracker before anyone sees it. A field you leave out gets checked for the first time by the person who receives the message.

2 Ask for a preview-only script

I used Claude Code. Use a code assistant your company approves, and start with fictional data. Don't give it production credentials just to see whether it can build the first version.

The prompt below describes what to build. It isn't a ready-to-run command. Replace the example rules with your own if they differ.

EXAMPLE

"Build a local preview script using the fictional cases in section 3. Accept an explicit report time. Apply the overdue and due-soon rules in section 1. Produce proposed messages, a review list, exclusions, and counts that account for every input row. Use stable item IDs and the fictional recipient mapping. Hold missing or conflicting records for review. Include the exact proposed text and reason for each message. Don't include any message-sending or tracker-writing code. Don't connect to live systems. Add tests for every expected result in the example. Explain how I can run the script and those tests on my machine."

Review the generated code and its dependencies before running it. If you can't tell whether it contacts a service or changes data, ask someone who can review those parts. A prompt saying "don't send" isn't proof that the code follows that instruction.

After the fictional tests pass, add only the approved read connection. Keep sending and tracker writes out of this version. An incomplete read, expired access, or missing result page should produce a visible failure. Store the report where your team permits it, because a read-only report can still contain private information.

WHY THIS MATTERS: with no sending code and no live connection, the worst a bug can do at this stage is print a wrong line on your screen.

3 Test records you already know the answer to

The records below are fictional. Set the report time to September 22, 2026, 9:00 AM in America/Los_Angeles. Treat dates as local calendar dates. "In progress" is active and "Done" is complete.

Unless a case says otherwise, use owner Alex Example, fake account alex-example, and last state change September 1 at 9:00 AM in the same time zone. Map that fake account to a fictional preview recipient, never a real colleague. Give each item a sample title and a local placeholder link.

- ☐ DEMO-101: In progress, due September 21. Expect one overdue message.
- ☐ DEMO-102: In progress, due September 22. Expect one due-soon message. It isn't overdue.
- ☐ DEMO-103: In progress, due September 29, last state change September 8 at 9:00 AM. Expect one due-soon message at both boundaries.
- ☐ DEMO-104: Done, due September 1. Exclude it as complete.
- ☐ DEMO-105: In progress, no due date. Hold for review.

- ☐ DEMO-106: In progress, due September 21, no owner. Hold for review.
- ☐ DEMO-107: In progress, due September 23, no state history. Hold for review.
- ☐ DEMO-108: In progress, due September 30. Exclude it as outside the window.
- ☐ DEMO-109: In progress, due September 25, last state change September 21 at 9:00 AM. Exclude it because the state changed within fourteen days.
- ☐ Add an exact copy of DEMO-101. Report one duplicate row, without proposing a second message for the item.

The expected totals are ten input rows, nine unique items, three proposed messages, three review items, three excluded items, and one duplicate row. If your output differs, leave the live connection off and find the mismatch.

Then change the duplicate's owner to another fake account and run again. DEMO-101 should move to review because the records conflict. Also try an unknown state and an unreadable input file. Neither should produce a successfully completed report that conceals the problem.

EXAMPLE

Your first run puts DEMO-102 in the overdue list. It's due on the report date, so the script is using "on or before" where the rule says "before." You caught it with nothing connected.

WHY THIS MATTERS: on live data a wrong count can look normal, because you don't know the right answer yet. These nine records have known answers, and a mistake like that one shows up before the script reads anything real.

4 Read the live preview and keep a review sheet

Once the read connection is approved, run it manually on a small agreed scope. Open every proposed item in the tracker and compare the owner, state, date, and wording. Check the review list too. Ask the owner about a discrepancy before deciding which value is wrong.

EXAMPLE

"Hey, the preview lists this item under your account, but I thought it had moved to another team. Can you confirm who owns it now? I've held the reminder until we sort that out."

Copy this sheet for each run:

- ☐ Run date, time zone, project scope, and rule version:
- ☐ Read completed successfully, or failure to investigate:
- ☐ Input rows / unique items / duplicate rows:
- ☐ Proposed messages / review items / excluded items:
- ☐ Item ID and the mismatch I found:
- ☐ Who will confirm or correct the record:
- ☐ Correction made and result of rerunning:
- ☐ Sending disabled and tracker writes absent: yes / no:

Save the reports according to your team's retention rules so you can compare runs. Repeat after an owner, date, or state changes. If those cases haven't occurred in the live data, test them with fictional records instead.

You can use a week of previews to observe the team's normal workflow, but elapsed time alone doesn't establish that the script works. Keep unresolved items out of the pilot, and rerun the examples whenever you change the rules.

WHY THIS MATTERS: every mismatch on that sheet is a message that would have gone to the wrong person or said the wrong thing.

5 Set the limits for a small pilot

Choose the people and items for the pilot, and get their agreement before enabling messages. Show them a sample message and explain whether replies will be copied anywhere. Part 4 covers that behavior.

Use the preview counts to choose a per-run message limit and a separate per-person limit. Write down what happens when either is exceeded. For an initial pilot, hold the proposed batch for review rather than sending a partial batch without telling you.

A higher count may reflect more overdue work or a wider query. It doesn't prove the data broke. Compare the items with the previous run before deciding whether the increase is expected.

- ☐ Every fictional case produces its expected result.
- ☐ The live read finishes without missing pages or hidden errors.
- ☐ Proposed recipients and item details match the source.
- ☐ Unresolved records stay out of the proposed batch.
- ☐ Participants, message limits, and reminder timing are agreed.
- ☐ A named person will review failures and can stop sending.
- ☐ The sending and reply controls in Parts 4 and 5 will be tested before use.

EXAMPLE

Fictional limits: at most twelve messages per run, at most two per person per day, and three days before the same item can be mentioned again. Anything over those numbers is held for the pilot owner to review before it sends.

WHY THIS MATTERS: the people in the pilot agreed to a set number of messages. A run that sends more without your review breaks that agreement.

Keep sending off until those checks are complete. You can work on the message wording in Part 4 without enabling delivery.

Part 4 covers message wording, one item per message, and how to match a reply to the right work item without guessing.